Test Plan and Test Cases: KitchenFlow Restaurant Order & Kitchen Display System

Document ID: KF-TPL-001 · Version 0.1 (draft) · 30 October 2026 · Author: Rathi · Related: KF-PDD-001, KF-SRS-001, KF-SAD-001, KF-DBD-001, KF-API-001, KF-LLD-001

1. Introduction

1.1 Purpose

This is document 7 of 7. It defines how KitchenFlow is shown to work: the strategy, the environments and data, the test cases (what to do and what must happen), the pass criteria, and the trace from every requirement to the tests that prove it. It also closes the document set: section 17 gathers the changes that documents 3 to 6 proposed to earlier documents into one register, so nothing is lost.

1.2 Scope

In scope: the backend, the database, the REST and streaming API, and the deployed stack (Compose and Nginx). The screens are covered only where a requirement states a behaviour (timing, readability, the demo). Out of scope: third-party penetration testing, multi-outlet operation, and real payment gateways.

1.3 How to use this document

The plan lives here; the executable tests live in the repository, and every test method carries its case ID in its name (LLD 6.4). A case lists the requirement IDs it proves, so a failing test points straight back to a requirement and a passing suite can be traced forward to coverage.

Family	Meaning	Section
UT	Unit tests of pure logic	4
AR	Architecture and static checks	5.2
DB	Database and migration tests	5.1
WS	Web-layer tests: envelope, strictness, content negotiation	6.5

Family	Meaning	Section
CT	Contract tests: one per row of every API case table	6.1
PC	Post-condition suites: what must (and must not) change after an operation	6.3
RM	Role matrix	6.4
CC	Concurrency and mutation tests	7
SS	Stream tests	8
RL, CA	Rate-limit and cache tests	9
SC	Security tests	10
RS	Resilience and fault tests	11
PF	Performance and load tests	12
AT	Acceptance scenarios (SRS section 12.2)	13
US, DM	Usability checks and the demo	14

Cases are written as Given, When, Then. Numbers in cases are the values to assert, taken from the documents (for example the 513.72 order of the API document).

2. Test strategy

2.1 Objectives

1. Prove the correctness invariants under concurrency: no duplicate, lost, oversold or stuck orders (SRS goal G1).
2. Prove the API contract, including every error case in the API document.
3. Prove real-time behaviour and recovery: screens update within 1 second and heal after a disconnect.
4. Prove protection and resilience: access control, limits, overload and fault behaviour.
5. Keep the proof cheap enough to run on every commit, and honest: each protection has a mutation variant that must fail when the protection is removed.

2.2 Risk-based emphasis

Risk	Impact	Where the effort goes
Duplicate or lost order under retries and concurrency	Very high	CC-01, CC-05, CC-08, AT-03
Overselling the last item	Very high	CC-02, AT-04
An order stuck in IN_KITCHEN, never READY	High	CC-04 with its mutation variant, AT-01, AT-06
Wrong money (rounding, tax, totals)	High	UT-01, UT-02, DB-04, the contract totals
Unauthorised access or privilege escalation	High	RM-01, SC-01 to SC-08, AT-08
Screens stale or silent after a fault	High	SS-01 to SS-14, AT-09, PF-02
Overload, abuse, retry storms	Medium	RL-01 to RL-12, PF-07, RS-04
Data left inconsistent after a failure	High	RS-01, RS-02, the consistency queries C1 to C7
Slow queries at volume	Medium	DB-09, PF-03 to PF-05
Silent drift of the documented contract	Medium	CT, the case-coverage and drift checks

2.3 Levels

Level	Purpose	Written in phase	Runs
Unit	Pure rules in milliseconds	With each class (LLD section 7)	Every commit
Web slice	Envelope, strict JSON, content negotiation, URL roles	Phase 1	Every commit

Level	Purpose	Written in phase	Runs
Integration and contract	Real PostgreSQL and the real migration; API case tables	Phases 1 to 3	Every commit
Concurrency	The invariants of SAD 6.2, repeated 50 times	Phases 2 and 3	Every commit (shortened) and nightly (full)
Streams	Delivery, recovery, limits	Phase 2	Every commit
Security and resilience	Attacks, faults, overload	Phases 3 and 4	Nightly and before release
Performance	NFR-PERF targets on 100,000 orders	Phase 4	Before release; a smoke version in CI
Acceptance	The 11 SRS scenarios end to end	Phases 2 to 4	Before release and before every demo

2.4 Principles

- **Test first.** Each build step in LLD section 7 begins with its failing tests.
- **Deterministic.** No Thread.sleep: asynchronous assertions use Awaitility with a timeout. Time comes from a controllable Clock; random data uses fixed seeds; bucket refill uses an injected time meter.
- **Independent.** Each test builds the data it needs and never relies on another test's leftovers or on execution order.
- **Real database.** Integration tests run the real Flyway migration, the real roles and real constraints; the database is never mocked.
- **Prove the protection.** For each lock, constraint or compare-and-set that exists for correctness there is a variant with it removed, and the test must then fail (LLD 6.2).
- **Flaky means broken.** A test that fails intermittently is a defect in the test or in the code. It is fixed within 2 working days or deleted; it is never retried until green.

2.5 Entry and exit criteria

Gate	Criteria
Start of a build step	The relevant design sections are read; the previous step is green; the step's tests are written and failing
End of phase 1	Unit and web-slice tests green; role matrix green for every implemented endpoint; the 4 schema checks and C1 to C7 clean on seed data
End of phase 2	AT-01, AT-05, AT-06, AT-09 and AT-11 pass; CC-03 and CC-04 pass with their mutation variants
End of phase 3	All CC cases pass 50 repeats; RL and CA cases pass; AT-02, AT-03, AT-04, AT-07, AT-08 and AT-10 pass
End of phase 4	PF-01 to PF-05 meet their thresholds on the 100,000-order dataset; every CI gate green; the demo script rehearsed twice
Release (demo or interview)	Every Must requirement has at least one passing test; all 11 acceptance scenarios pass; no open Sev 1 or Sev 2 defect; coverage at least 70 percent on service and domain code; the traceability matrix (section 15) has no empty row; the change register (section 17) is applied or consciously deferred

2.6 Defect handling

Severity	Meaning	Examples	Rule
Sev 1	Data corruption, duplicate or oversold order, security bypass	Two payments recorded; a Kitchen user changes another station	Fix before anything else; add a regression test first
Sev 2	A Must requirement is not met and there is no workaround	Order never reaches READY; wrong GST total	Fix before the phase gate

Severity	Meaning	Examples	Rule
Sev 3	A Should or Could requirement is not met, or a documented case is wrong	Missing audit field; wrong field code	Fix before release or defer in writing
Sev 4	Cosmetic or documentation	Typo in a message	Fix when convenient

Every defect gets a failing test before the fix and keeps that test. Defects are tracked as GitHub issues labelled with the severity and the case ID that found them.

2.7 Metrics and reporting

At the end of each phase, one page: tests run and passed per family; line coverage of service and domain code (JaCoCo); requirement coverage from section 15; concurrency repeat results (failures must be zero); mutation variants that failed as intended; performance numbers against thresholds; open defects by severity. The same numbers decide the release criteria in 2.5.

2.8 Roles

One developer writes code and tests. A reviewer (a senior, a friend or a mentor) reads this plan, the concurrency tests and the demo script, and watches one full acceptance run. Before each merge the developer applies a self-review checklist: tests first, case IDs in names, no sleeps, no shared data, every new endpoint in the role table and the case files.

3. Environments, tools and data

3.1 Environments

Environment	Composition	Used for
Local development	Application in the IDE or in Compose, PostgreSQL in Compose, profile dev	Writing and running unit, slice and integration tests

Environment	Composition	Used for
CI	GitHub Actions runner, Testcontainers for PostgreSQL	Every commit: build, tests, architecture checks, coverage gate, container build, smoke test
Performance	A laptop-class machine (4 cores, 8 GB) running the full Compose stack, profile loadtest, 100,000-order dataset	PF cases; the reference machine named in the SRS
Demo	Compose with Nginx, seed data, normal limits	Acceptance run and the 5-minute demo

3.2 Tools

Tool	Use
JUnit 5, AssertJ	Test framework and assertions
Mockito	Only for collaborators outside the unit under test; never for the database
Spring Boot Test, MockMvc	Web slice tests
RestAssured	Black-box tests against the running application
Testcontainers	Real PostgreSQL (and Nginx for the proxy checks)
Awaitility	Waiting for asynchronous effects with a timeout
ArchUnit	Architecture rules
JaCoCo	Coverage and the 70 percent gate
k6	Load, stress and smoke scenarios
openapi-diff	Compatibility of the generated OpenAPI against the last release
Bruno or Postman	Manual exploration and the demo collection
Secret scanner and dependency scan	Repository and library hygiene (SC-15, SC-16)

3.3 Test data

Dataset	Content	Used by
Built per test	TestData builders: user(role), station, category, item(price, stations, stock), modifier group, combo, order in a chosen status	Almost every integration test
Seed (demo)	DBD section 10: 9 users, 4 stations, 5 categories, 30 items, 4 modifier groups, stock for 4 items	Acceptance run, demo, seed tests
Known report dataset	About 40 hand-built orders over 3 business days, with expected report figures worked out by hand and stored beside it	Report ground truth (CT for R-01 to R-05, AT-10)
Performance dataset	The 100,000-order generator of DBD 10.6	DB-09, PF cases

Isolation. Between test classes a cleaner truncates every table except the Flyway history and the reference data, as the schema owner (kf_app has no TRUNCATE). Inside a class every test creates uniquely named data. Database-backed classes run one after another; unit tests run in parallel.

3.4 Time, randomness and configuration

- A MutableClock bean replaces the system clock in the test profile and can be advanced, so lockout windows, the recall window, token expiry and the business-day cutoff are tested without waiting.
- Rate-limit buckets take an injected time meter, so refill is tested instantly (RL-01).
- The test profile shortens intervals that are fixed in production: the stream heartbeat (200 ms) and token lifetime. This needs the sse heartbeat to be a configuration property (change 14 in section 17).
- Generators and load scripts use fixed seeds, so a failure can be reproduced.

3.5 Accounts, secrets and artefacts

Test users and passwords come from the TestData builders and the test profile; the JWT key in tests is a constant that is never used elsewhere. No real secret appears in the repository (SC-15). Every run keeps its reports as build artefacts: surefire and failsafe results, the JaCoCo report, k6 summaries, and the saved EXPLAIN plans.

4. Unit test cases

Unit tests need no Spring context. The branch coverage target for Money, PricingService, BusinessDayService, the two state machines and SlaCalculator is 100 percent.

4.1 UT-01 Money arithmetic and rounding

Case	Input	Expected
Tie rounds up	0.10 at 5.00 percent (0.005)	0.01
Tie rounds up	0.30 at 5.00 percent (0.015)	0.02
Tie rounds up	0.50 at 5.00 percent (0.025)	0.03
Below a tie	12.34 at 18.00 percent (2.2212)	2.22
Plain GST	99.00 at 18.00 percent	17.82
Plain GST	378.00 at 5.00 percent	18.90
Zero rate	250.00 at 0.00 percent	0.00
Largest values	99999.99 at 28.00 percent (27999.9972)	28000.00
Multiplication	189.00 times 2	378.00
Addition	99.00 plus 17.82	116.82
Comparison	10.00 compared with 9.99	Greater
Wrong scale rejected	A Money built from 1.5	IllegalArgumentException
Parse	Money.parse of 1.50 succeeds; of 1.5 fails	As stated

4.2 UT-02 Order pricing vectors

Each vector gives lines as unit price, modifier total, quantity and GST rate. The same vectors are inserted through SQL in DB-04 to prove the database CHECK constraints agree with Java.

Vector	Lines	Expected totals
A (the API example)	169.00 with Hot 0.00 and Extra cheese 20.00, quantity 2, 5.00 percent; 69.00 with Large 30.00, quantity 1, 18.00 percent	Line 1: subtotal 378.00, tax 18.90, total 396.90. Line 2: subtotal 99.00, tax 17.82, total 116.82. Order: subtotal 477.00, tax 36.72, total 513.72
B (mixed rates)	99.00 x 1 at 5 percent; 49.00 x 3 at 5 percent; 129.00 x 1 at 18 percent	Taxes 4.95, 7.35 and 23.22; subtotal 375.00, tax 35.52, total 410.52
C (tax is rounded per line)	Two lines of 0.10 x 1 at 5 percent	Each line tax 0.01, so order tax 0.02 and total 0.22 (rounding the order once would give 0.01)
D (negative modifier)	99.00 with a modifier of minus 10.00, quantity 2, 5 percent	Subtotal 178.00, tax 8.90, total 186.90
E (the ceiling)	99999.99 x 50 at 28 percent	Subtotal 4999999.50, tax 1399999.86, total 6399999.36; fits NUMERIC(12,2)

4.3 UT-03 and UT-04 JSON for money and instants

Case	Input	Expected
Valid money string	129.00 as a string	Accepted
Number instead of string	129.00 as a JSON number	Type mismatch, mapped to 400 MALFORMED_REQUEST
Wrong scale	129.5 and 129.500 as strings	INVALID_SCALE
Not a number	abc	INVALID_FORMAT

Case	Input	Expected
Negative where unsigned	-5.00	INVALID_FORMAT; accepted by the signed variant
Too many integer digits	12345678901.00	INVALID_FORMAT
Blank	An empty string	REQUIRED
Money out	Money 5.00	The string 5.00
Instant with 0 ms	2026-10-30T09:15:30Z	Written as 2026-10-30T09:15:30.000Z
Instant with 100 ms	2026-10-30T09:15:30.1Z	Written as 2026-10-30T09:15:30.100Z

4.4 UT-05 Business day (cutoff 04:00, Asia/Kolkata)

Instant	Expected business day
2026-10-31 03:59:59 IST	2026-10-30
2026-10-31 04:00:00 IST	2026-10-31
2026-10-30T22:30:00Z (04:00 IST on 31 October)	2026-10-31
2026-10-31 00:00:00 IST	2026-10-30
2026-10-31 23:59:59 IST	2026-10-31
With the cutoff changed to 05:00: 2026-10-31 04:30 IST	2026-10-30
With the cutoff at 00:00: any time on 31 October	2026-10-31

4.5 UT-06 Order state machine

The test generates all 180 combinations (6 from-states, 6 to-states, 5 roles including SYSTEM) and compares each with an independently written expected table. No rule for the pair gives 409; a rule that excludes the role gives 403.

Transition	Allowed roles	Other roles	Notes
CREATED to PAID	CASHIER, MANAGER	403	
PAID to IN_KITCHEN	SYSTEM	403	
IN_KITCHEN to READY	SYSTEM	403	
READY to IN_KITCHEN	KITCHEN, MANAGER	403	Recall
READY to COMPLETED	CASHIER, MANAGER	403	
CREATED to CANCELLED	CASHIER, MANAGER, SYSTEM	403	
IN_KITCHEN to CANCELLED	MANAGER	403	A Cashier gets 403, not 409
READY to CANCELLED	None	409 for every role	
Any state to COMPLETED or CANCELLED when already final	None	409 for every role	
Skips and repeats, for example CREATED to IN_KITCHEN, CREATED to READY, PAID to CANCELLED, CREATED to CREATED	None	409 for every role	

4.6 UT-07 Ticket transitions

Transition	Expected
NEW to IN_PROGRESS	Allowed; startedAt set
IN_PROGRESS to DONE	Allowed; doneAt set
DONE to IN_PROGRESS (recall)	Allowed; doneAt cleared; recallCount plus 1; startedAt unchanged

Transition	Expected
NEW or IN_PROGRESS to CANCELLED	Allowed; cancelledAt set
NEW to DONE; DONE to NEW; DONE to CANCELLED; IN_PROGRESS to NEW; DONE to DONE; any move from CANCELLED	INVALID_TRANSITION

After every legal transition in the integration variant, the row is written and the CHECK constraints ck_ticket_started, ck_ticket_done and ck_ticket_cancel hold.

4.7 UT-08 SLA colour (target 300 seconds, amber 80, red 100)

The colour compares exact products, not a truncated percent: red when age x 100 is above red x target. The 301-second vector guards that rule, because a truncated percent would read 100 and stay amber.

Age in seconds	Percent	Colour
0	0	GREEN
239	79	GREEN
240	80	AMBER
299 and 300	99 and 100	AMBER
301	100	RED
450	150	RED
DONE ticket created 09:00, done 09:03, read at 09:30	60	GREEN (the age stops at doneAt)
Thresholds changed to amber 50, red 90: age 150 and age 271	50 and 90	AMBER and RED
Target of 1 second: age 1 and age 2	100 and 200	AMBER and RED

4.8 UT-09 Password policy

Password	Expected
7 bytes	TOO_SHORT
8 bytes	Accepted
72 bytes	Accepted
73 bytes	TOO_LONG
40 characters that are 2 bytes each (80 bytes)	TOO_LONG, although 40 characters
Equal to the username	WEAK_PASSWORD
Equal to the current password (change-password)	WEAK_PASSWORD
On the common-password list, for example password123	WEAK_PASSWORD

4.9 UT-10 OrderValidator

Input	Expected error
No lines; 31 lines	TOO_FEW; TOO_MANY on lines
Quantity 0; quantity 51	OUT_OF_RANGE on lines[i].quantity
Both itemId and comboId; neither	ONE_OF_REQUIRED on lines[i]
Unknown itemId	UNKNOWN_ID on lines[i].itemId
Unavailable item; inactive item	ITEM_UNAVAILABLE
Only unavailable items	Top-level code ITEM_UNAVAILABLE
An unavailable item and quantity 0 together	Top-level VALIDATION_FAILED with both errors
Modifier of another item	MODIFIER_NOT_ALLOWED
Inactive modifier	MODIFIER_UNAVAILABLE
The same modifier id twice	DUPLICATE_VALUE
Group with minimum 1 and no selection	MODIFIER_MIN (message names the group)

Input	Expected error
SINGLE group with two selections; MULTIPLE group (maximum 3) with four	MODIFIER_MAX
modifierIds on a combo line	NOT_APPLICABLE
Note of 201 characters; note with a control character	TOO_LONG; INVALID_CHARACTERS
orderType missing; DRIVE_THRU	REQUIRED; INVALID_ENUM
A line with quantity 0 that also names an unknown modifier	Only the quantity error: a malformed line is not checked against the menu

4.10 UT-11 to UT-17 Other pure logic

ID	Subject	Cases
UT-11	Idempotency hash	Same body with keys in another order: equal. Extra whitespace: equal. Two lines swapped: different. One quantity changed: different
UT-12	Cursor codec	Round trip; changed payload, changed signature, truncated value and wrong key are all rejected
UT-13	Rate-limit classification	POST /orders, PUT /orders/9, PATCH /orders/9/rush, POST /orders/9/payments give ORDER_WRITE; POST /tickets/5/bump gives TICKET_ACTION; GET /menu and GET /orders give READ; GET /reports/sales and GET /audit give REPORT; POST /admin/items and PATCH /items/3/availability give ADMIN_WRITE; GET /board gives BOARD_READ keyed by address; GET /stream/stations/1 gives SSE_CONNECT keyed by user; GET /stream/board gives SSE_CONNECT keyed by address; POST /auth/refresh gives REFRESH; health, static files and POST /auth/login give no class in the filter
UT-14	Order number	(30 Oct 2026, 17) gives KF-20261030-0017; token 9999 gives -9999; token 10000 gives KF-20261030-10000
UT-15	Configuration rules	Each range of API 7.2; amber at or above red rejected; business_day_cutoff outside 00:00 to 08:00 rejected; amber and red validated against values already stored

ID	Subject	Cases
UT-16	Ticket grouping	A burger on Fry and Assembly plus a cola on Beverage gives three tickets, the burger on both of its tickets; target seconds is the largest on each ticket; a combo expands into its components; a line with no active station raises IllegalStateException
UT-17	Cancel policy and stock maths	Which role may cancel in which state; the stock difference of an edit, per item in ascending id order

5. Database and architecture tests

5.1 Database tests

ID	Test	Expected	Proves
DB-01	Run roles.sql and V1 on an empty container	No error; Flyway validate passes	NFR-REL-03
DB-02	The four schema checks of DBD 11.4	Zero rows each	DD-03, DD-04
DB-03	Insert a duplicate for every UNIQUE constraint: uq_orders_number, uq_orders_day_token, uq_payment_order, uq_idem_user_key, uq_ticket_order_station, uq_app_user_username (also with other letter case), uq_station_name, uq_category_name, uq_menu_item_name (other case), uq_combo_name, uq_modifier_name, uq_rt_hash, uq_ol_order_line, the refund-once index	SQLSTATE 23505 each	DR-04

ID	Test	Expected	Proves
DB-04	For each CHECK, insert a violating row and a boundary-valid row: ck_orders_money, ck_orders_paid, ck_orders_cancel, ck_ol_qty (0 and 51 fail; 1 and 50 pass), ck_ol_sub, ck_ol_tax (off by 0.01), ck_ol_total, ck_ol_target (both or neither), ck_stock_qty, ck_ticket_started, ck_ticket_done, ck_ticket_cancel, ck_payment_amount, ck_payment_cash, ck_mg_range, ck_mg_single, ck_category_gst (28.01 fails, 28.00 passes), ck_item_price, ck_user_role; plus the UT-02 vectors	23514 for violations; boundary rows accepted	DR-03, BR-02, BR-03
DB-05	Insert an order line for a missing order; delete a referenced menu item	23503	DR-03, DR-05
DB-06	As kf_app: UPDATE and DELETE on audit_event; any DDL; DELETE on orders; TRUNCATE. Then DELETE on item_station	The first four fail with permission denied; the last succeeds	FR-AUD-02, DR-09
DB-07	As the schema owner, UPDATE and DELETE on audit_event	The trigger raises the append-only exception	FR-AUD-02
DB-08	Insert 120 tickets through JPA and count the batches	Ids step by 50; at most 3 insert batches, so batching works (DD-02)	SAD 5.5
DB-09	EXPLAIN (FORMAT JSON) for the statements of DBD 7.2 on the 100,000-order dataset	The named index appears; no sequential scan on orders, ticket or audit_event; no Sort for S6	DR-07
DB-10	Consistency queries C1 to C7 after (a) the acceptance run, (b) the generator, (c) the concurrency suite	Zero rows each	BR-01, BR-06, BR-07, DD-09

ID	Test	Expected	Proves
DB-11	Run the generator on an empty database	Under 2 minutes; 100,000 orders; at least 500,000 audit rows; about 2 tickets per order; 97 percent completed, plus or minus 1 point; refuses a non-empty database; the same seed gives the same total of order totals	FR-CFG-03
DB-12	Run the seed on an empty database	Under 60 seconds; 30 items, 9 users, 4 stations, all configuration keys; C6 clean; refuses a non-empty database	FR-CFG-02, BR-09
DB-13	100 sessions call S1 for one business day at once	Values 1 to 100, each once, no gap	BR-01
DB-14	Timestamps round-trip in UTC; the kf_app session time zone is UTC	Equal instants	DR-02, BR-18
DB-15	Run S1 to S3, S5 (including the amount condition), S6 (rush first, then oldest), S7, S8 and S9 (SKIP LOCKED) against known data	Documented results	DBD 8, LLD 8.2
DB-16	Run S10 to S14 on the known report dataset	Equal to the hand-computed figures	FR-RPT-01 to 05
DB-17	Backup and restore drill: dump, restore to a scratch database, run C1 to C7 and compare row counts	Zero rows and equal counts	DBD 12

5.2 Architecture and static checks

ID	Check	Tool
AR-01	api packages do not use infrastructure packages	ArchUnit
AR-02	domain packages use no web or servlet types	ArchUnit
AR-03	The ten feature modules have no dependency cycles	ArchUnit
AR-04	order does not depend on kitchen	ArchUnit
AR-05	@Transactional appears only in application packages	ArchUnit
AR-06	EndpointRuleCoverageTest: every controller mapping has an explicit URL rule, and the public rules match the endpoint index of the API document	Test over the handler mappings
AR-07	CaseCoverageTest: every row of every case table in the exported API document has a case in the YAML files	Script over the exported Markdown
AR-08	OpenAPI drift: the generated definition equals the API document's endpoint index (IDs, methods, paths, roles, classes)	Script
AR-09	Compatibility: no removed or retyped field against the last release	openapi-diff
AR-10	No double or float field in domain packages	ArchUnit
AR-11	No call to Instant.now() or LocalDate.now() outside ClockConfig	ArchUnit
AR-12	No System.out and no printStackTrace in main code	ArchUnit
AR-13	Formatting and lint rules pass; coverage at least 70 percent on service and domain code	CI gate

6. API contract and integration tests

6.1 Case source and format (CT)

Every row of every case table in the API document is one test. Its ID is the endpoint ID plus the row number in that endpoint's case table, counted from 1 in document order: O-01.14 is the fourteenth row of the O-01 table. The cases live in YAML files, one per endpoint, under src/test/resources/cases. A generic runner (a JUnit TestFactory) builds the fixtures, sends each request to the running application with RestAssured, and checks the response and the database. The expected values are copied from the API document; a change to a case table and the matching YAML change in the same commit.

```
endpoint: O-01
cases:
  - id: O-01.09
    title: item unavailable
    given:
      users: [ cashier1 ]
      items:
        - { name: Cola, price: '69.00', gst: '18.00', stations: [ Beverage ],
available: false }
    request:
      as: cashier1
      method: POST
      path: /orders
      body:
        orderType: TAKEAWAY
        lines: [ { itemId: '$item.Cola', quantity: 1 } ]
    expect:
      status: 422
      code: ITEM_UNAVAILABLE
      errors: [ { field: 'lines[0].itemId', code: ITEM_UNAVAILABLE } ]
    post:
      orders: 0
      stock: unchanged
      audit: none
```

Symbolic references such as $item.Cola, $user.cashier1 and $order.first are resolved by the runner. A row marked n/a in the API tables (a race between two requests) has no YAML case of its own; its entry points to the concurrency case that covers it, for example see: CC-05.

For every response the runner also checks:

- **Errors:** the whole envelope (timestamp, status, code, message, path, correlationId, errors, details); no stack trace or class name in any field; X-Correlation-Id equal to correlationId; Cache-Control no-store; Retry-After on 429 and 503.
- **Successes:** the body validates against the generated OpenAPI schema, money fields are strings with two decimals, instants have three millisecond digits, and no undocumented field appears.

6.2 Fixtures and extra assertions by endpoint group

Group	Endpoints	Fixtures	Assertions beyond status and code
Authentication	A-01 to A-05	A user per role, one inactive user, the MutableClock	failed_attempts and locked_until in the database; Retry-After seconds; cookie flags HttpOnly, Secure, SameSite=Strict and path /api/v1/auth; no password hash in any response or log
User admin	U-01 to U-05	Two admins, a kitchen user with a station	Refresh tokens revoked on deactivation; USER_UPDATED audit; SELF_LOCKOUT and LAST_ADMIN
Stations and configuration	S-01 to S-03, C-01, C-02	A station with open tickets; an item whose only station is the one being deactivated	Cache evicted at once; CONFIG_CHANGED sent; audit
Menu	M-01 to M-21	Categories, items, modifier groups with minimum and maximum, an item that was ordered, a combo	menu_version incremented; ETag changes; item version incremented; a failed delete leaves every dependent row in place

Group	Endpoints	Fixtures	Assertions beyond status and code
Inventory	I-01 to I-03	A tracked item with stock, an untracked item	Quantity; availability flips at zero; adjustment row and audit
Orders	O-01 to O-08	Menu with SINGLE and MULTIPLE groups, a tracked item, an unavailable and an inactive item, a combo, orders in every status	PC-01 to PC-03
Payments	P-01 to P-06	Orders in CREATED, IN_KITCHEN and CANCELLED; items on one, two and three stations; a deactivated station	PC-04, PC-13; bill rows sum to the order totals
Kitchen	K-01 to K-07	An IN_KITCHEN order with three tickets; bound and unbound users; the MutableClock for the recall window	PC-05 to PC-07
Board	B-01	Orders in every status	Only tokens and times; nothing personal
Reports and audit	R-01 to R-05, AU-01	The known report dataset; an audit dataset of 500 rows	Figures equal the stored expected values; range limits; bulkhead
Platform	H-01 to H-03	A stopped database container	Health DOWN with 503; the docs endpoints follow the profile

6.3 Post-condition suites (PC)

Status codes are not enough: each state-changing operation also has to leave the system in exactly the right state, and a failed operation has to leave nothing behind.

ID	Operation	After success, all of this is true	After any failure, nothing changed
PC-01	O-01 create order	One order row with status CREATED and version 0; token is the previous token of that business day plus 1 and the order number matches KF-yyyyMMdd-NNNN; line and modifier snapshots equal the response; totals satisfy C1; tracked stock is reduced by the quantities; exactly one ORDER_CREATED audit record by the caller; with a key, the idempotency row holds the response and the order id; no stream event is sent	No order, line or modifier rows; stock and the token counter unchanged (the next order takes the token the failed one would have had); no audit; no idempotency row
PC-02	O-04 edit order	Same id, token, number and business day; lines replaced; totals recomputed; stock reflects the difference; version plus 1; ORDER_EDITED audit; an identical edit changes nothing, not even the version	Old lines, totals, stock and version intact
PC-03	O-05 cancel order	Status CANCELLED with reason, canceller and time set; NEW and IN_PROGRESS tickets CANCELLED and DONE tickets untouched; stock restored only when the order was CREATED; REFUND_PENDING refund and payment status only when it was paid; token gone from the board; ORDER_CANCELLED audit; TicketCancelled and OrderStatusChanged events after commit	Order, tickets, stock, refund unchanged

ID	Operation	After success, all of this is true	After any failure, nothing changed
PC-04	P-01 pay	Order IN_KITCHEN with paidAt set; one payment row whose amount equals the total (and changeDue for cash); exactly one ticket per distinct active station among the items, each with its own items, targetSeconds equal to the largest on it, rush and business day copied, status NEW; audit PAYMENT_RECORDED, ORDER_PAID and ORDER_IN_KITCHEN; TicketCreated per ticket and OrderStatusChanged after commit; the board lists the token as preparing	Order still CREATED at its old version; no payment, no tickets, no audit, no events
PC-05	K-03 start	Ticket IN_PROGRESS, startedAt set, version plus 1, TICKET_STARTED audit, TicketChanged event	Ticket unchanged
PC-06	K-04 bump	Ticket DONE with doneAt; when it was the last open ticket the order is READY with readyAt, ORDER_READY is audited exactly once in the order's life and the board shows the token as ready; otherwise the order stays IN_KITCHEN	Ticket and order unchanged
PC-07	K-05 recall	Ticket IN_PROGRESS with doneAt cleared and recallCount plus 1; a READY order returns to IN_KITCHEN with readyAt cleared and the token back in preparing; TICKET_RECALLED audit	Ticket and order unchanged
PC-08	M-17 availability	Item availability and version changed; menu_version plus 1; ITEM_AVAILABILITY_CHANGED audit; the next GET /menu shows the new state with a new ETag. A request that changes nothing leaves version, menu_version and audit untouched	Unchanged
PC-09	I-02 stock adjustment	Quantity plus delta; adjustment row with actor and reason; STOCK_ADJUSTED audit; availability flips when the quantity crosses zero	Unchanged

ID	Operation	After success, all of this is true	After any failure, nothing changed
PC-10	Admin catalogue changes (M-03 to M-16, M-19 to M-21, S-02, S-03)	One audit record per change with the actor; menu_version plus 1 for menu changes and the menu cache evicted; a delete removes the item and all its dependent rows	A failed change, including a delete refused by a foreign key, leaves every row in place
PC-11	User changes (U-02, U-04, U-05, A-05)	The password is stored only as a BCrypt hash; refresh tokens revoked where specified; audit rows	Unchanged
PC-12	C-02 configuration	All keys applied; cache evicted; CONFIG_CHANGED sent; audited once	None applied
PC-13	P-06 refund completion	Refund and payment both REFUNDED in one transaction	Both unchanged
PC-14	The unpaid-timeout job	CREATED orders older than the timeout become CANCELLED with UNPAID_TIMEOUT, actor SYSTEM, stock restored, audit written; paid orders untouched; an order locked by an in-flight payment is skipped	Not applicable

6.4 Role matrix (RM)

ID	Case	Expected
RM-01	Every endpoint of API section 4 called with a valid request as anonymous, CASHIER, KITCHEN, MANAGER and ADMIN (73 x 5 combinations)	Anonymous gets 401 except on public endpoints; allowed roles never get 401 or 403; every other role gets 403. The expected table is written by hand from the API index, not generated from the security configuration

ID	Case	Expected
RM-02	Station binding: a Kitchen user bound to Fry; multi1 bound to Fry and Grill; a Manager	Fry user: K-01 and X-01 for Fry succeed, for Grill 403; a Grill ticket on K-02 to K-06 gives 403 after the lookup. multi1: Fry and Grill succeed, Assembly 403. Manager: all stations succeed
RM-03	State-dependent role: cancel an IN_KITCHEN order	Cashier 403; Manager 200. Cashier cancelling a CREATED order: 200
RM-04	Token faults: expired, wrong signature, algorithm none, no role claim, unknown role value, wrong key	401 each, in the standard envelope
RM-05	A user deactivated while holding a valid token	The token still works until it expires; refresh and login fail at once; the user's stream ends at expiry

6.5 Web-layer tests (WS)

ID	Case	Expected
WS-01	One test for every row of the exception table and the constraint table in LLD 3.3, using real violations where possible (duplicate station name, delete a used item, second payment)	The documented status, code and field code, always in the envelope
WS-02	Strict JSON: an unknown property; a number where a string is expected; the string 2 where a number is expected; 2.5 for a quantity; null for a primitive; duplicate property names; trailing text	400 MALFORMED_REQUEST each, naming the offender where there is one (duplicate detection needs the Jackson option in change 15)
WS-03	Unknown query parameter on every GET endpoint; an empty filter value; a valid filter	400; 422 INVALID_FORMAT; 200

ID	Case	Expected
WS-04	Wrong Content-Type; Accept of application/xml; DELETE on /orders; HEAD on a GET endpoint; a CORS preflight	415; 406; 405 with Allow; same headers with no body; the allowed methods and headers
WS-05	Correlation id: a valid client value; a value with a space, 65 characters, or a newline	Valid one echoed; invalid ones replaced by a UUID; the header present on every response, including 401, 413 and 429 produced by filters
WS-06	A body of 1 MB plus one byte, with Content-Length and chunked	413 PAYLOAD_TOO_LARGE
WS-07	Pagination on every list endpoint: defaults; size 100; size 101; page minus 1; unknown sort field; bad direction; page beyond the last; cursor together with page; a tampered cursor	20 per page; 200; 422; 422; 422 UNKNOWN_SORT_FIELD; 422 INVALID_ENUM; 200 empty; 400; 400
WS-08	Caching headers	no-store on authenticated responses; on /menu an ETag, 304 for a matching If-None-Match and a changed ETag after a change
WS-09	Strings: padded with spaces; blank; with a control character	Trimmed; REQUIRED; INVALID_CHARACTERS
WS-10	Formats in every success body	Money strings with two decimals; instants with milliseconds and Z; no null where a value is documented

6.6 Idempotency and version contract (CT-ID, CT-VER)

ID	Case	Expected
CT-ID-01	Same key and payload sent twice	Second response is the first one, with Idempotency-Replayed true; one order
CT-ID-02	Same key, different payload	422 IDEMPOTENCY_KEY_REUSED with details.firstSeenAt; nothing created

ID	Case	Expected
CT-ID-03	Malformed key (81 characters; a space)	422 INVALID_FORMAT on the header
CT-ID-04	Replay after the order was paid	The original snapshot (status CREATED), not the current state
CT-ID-05	Another user sends the same key	An independent order
CT-ID-06	No key, same body twice	Two orders
CT-ID-07	Same key after 24 hours (clock advanced, purge job run)	A new order
CT-ID-08	First attempt fails with 422, second attempt with the same key and a corrected body	201: nothing was stored by the failure
CT-VER-01	A stale expectedVersion on every endpoint that takes one	409 STALE_VERSION with details.current equal to the current representation
CT-VER-02	Repeat start, bump, cancel, complete and refund completion	409 INVALID_TRANSITION whose details.currentStatus already equals the target state

7. Concurrency and mutation tests

These tests are the heart of the project: they prove the claims of SAD section 6 about duplicates, overselling and stuck orders. Rules for all of them:

- A shared helper releases the threads together with a latch (LLD 6.2). Each thread uses its own user so the per-user rate limits (30 order writes a minute) do not interfere; the test profile also raises the limits.
- Every case runs 50 times in the nightly build and 5 times on each commit. The failure budget is zero: a single failure in 50 is a defect.
- Waiting uses Awaitility with a 30-second timeout and never sleeps.
- After each case the consistency queries C1 to C7 must return zero rows.
- A mutation variant (marked m) removes one protection through a seam or a scratch schema and must make its test fail, which proves the test can see the bug.

ID	Given	When	Then	Proves
CC-01	A valid order and one Idempotency-Key	50 threads POST /orders with the same key and body	One order row; all 50 responses are 201 with the same body; 49 carry Idempotency-Replayed true; one ORDER_CREATED audit; the token counter moved by 1	NFR-CONC-01, AT-03, BR-15
CC-01m	The same, with the insert-first step replaced by check-then-insert and a barrier between the two	The same	More than one order exists in every repeat	The test detects a broken idempotency
CC-02	A tracked item with stock 10	100 threads each order one unit	Exactly 10 get 201 and 90 get 409 OUT_OF_STOCK; stock is 0 and the item is unavailable; a sampler reading the stock every 5 ms never sees a value below 0	NFR-CONC-02, AT-04, BR-12
CC-02m	The conditional UPDATE replaced by read, check, write	The same	More than 10 orders succeed in at least one repeat, or the CHECK constraint fires	The test detects overselling
CC-03	An IN_PROGRESS ticket at version v	Two users bump it at once with expectedVersion v; and the same with start on a NEW ticket	Exactly one 200; the other 409 (STALE_VERSION, or INVALID_TRANSITION when it saw the new state); the ticket is DONE once with one TICKET_BUMPED audit	NFR-CONC-03, AT-05, FR-KDS-07
CC-04	An IN_KITCHEN order whose last two tickets are IN_PROGRESS at different stations	Both are bumped at once	Both return 200; exactly one response has becameReady true; the order is READY; ORDER_READY is audited once; the board shows the token as ready. A variant with 3 tickets and 3 threads gives the same	NFR-CONC-04, AT-06, BR-06
CC-04m	The order lock replaced by a no-op, and a barrier that holds each transaction until both have updated their ticket	The same	The order is left IN_KITCHEN in every repeat (stuck), which is the bug that the lock prevents	The test detects the lost READY

ID	Given	When	Then	Proves
CC-05	A CREATED order	20 threads pay it at once	One 201; 19 get 409 PAYMENT_ALREADY_RECORDED; one payment row; one ticket per station, created once; ORDER_PAID audited once	FR-PAY-03
CC-05m	The compare-and-set replaced by read-check-write, and uq_payment_order dropped in a scratch schema	The same	More than one payment row appears	The test detects a double payment
CC-06	A CREATED order, 100 rounds	Pay and cancel start together in each round	In each round exactly one wins; the final state is either IN_KITCHEN with a payment and tickets, or CANCELLED with stock restored and neither; never both	BR-05, SAD 6.2
CC-07	A CREATED order, 100 rounds	An edit and a payment (with the old amount) start together	Either the edit wins and the payment gets 422 AMOUNT_MISMATCH, or the payment wins and the edit gets 409; the stored payment amount always equals the order total	FR-ORD-06, FR-PAY-04
CC-08	An empty business day	20 users create 10 orders each, together	Tokens are exactly 1 to 200 with no gap and no repeat; order numbers are unique; C7 is clean	BR-01
CC-09	An unpaid order older than the timeout, 50 rounds	The cancel job and a payment start together	Either the payment wins and the job skips the row, or the job wins and the payment gets 409 with stock restored; never a paid and cancelled order	FR-ORD-12
CC-10	Two transactions forced into a deadlock by a test hook	They run through DeadlockRetry	Both succeed within 2 retries and kf_db_lock_retry_total rises; a third consecutive deadlock gives 503 SERVICE_BUSY. Separately, a session holding the order row for 3 seconds makes a bump return 503 SERVICE_BUSY at about 2 seconds with no retry	SAD 6.5

ID	Given	When	Then	Proves
CC-10b	Three tracked items	50 orders request them in different line orders at once	No deadlock (retry counter 0); every order succeeds or gets OUT_OF_STOCK	Ascending lock order, SAD 6.1
CC-11	An item and 50 creates in flight	The item is made unavailable during the run	Each order either committed with the item before the change or was rejected with ITEM_UNAVAILABLE; no order created after the change contains it	BR-04, FR-MENU-04
CC-12	Two active admins	Each deactivates the other at once, 20 rounds	One succeeds; the other gets 422 LAST_ADMIN; at least one admin stays active	FR-AUTH-05
CC-13	One refresh cookie	Two threads refresh at once; then a third attempt 11 seconds later (clock advanced)	One 200 and one 401 without revoking; the later attempt gets 401 and revokes the family, so the new token fails too	FR-AUTH-07
CC-14	One user	10 wrong-password logins at once	The user ends locked; failed_attempts equals the number of 401 responses, with no lost update; the rest are 429	FR-AUTH-06
CC-15	A tracked item	Creates and cancels (while CREATED) run together	At the end initial stock equals the quantity plus the units on live orders; the quantity never exceeds the initial stock	BR-12

8. Stream tests

Stream tests use a real HTTP client that reads the event stream, with the heartbeat interval set to 200 ms and token lifetime to 2 seconds in the test profile. Client-side behaviours are tested in JavaScript with Vitest.

ID	Case	Expected	Proves
SS-01	A bound Kitchen user connects to X-01	200 with Content-Type text/event-stream, Cache-Control no-cache and X-Accel-Buffering no; the first event is hello with serverTime, stream, heartbeatSeconds and expiresAt equal to the token expiry	FR-KDS-05, API 12
SS-02	10 open streams on 5 stations; 100 orders paid one after another	Every payment produces TICKET_CREATED on each involved station within 1 second (the p95 over the run is checked again in PF-02)	NFR-PERF-02
SS-03	A stream left idle	A comment line arrives every heartbeat interval; the stream stays open for 3 intervals	SAD 9.4
SS-04	A station stream is open; the client is dropped; 3 orders are paid; the client reconnects and fetches the snapshot	Within 3 seconds the screen state equals the live queue, in the order of BR-10, with no duplicate	AT-09, FR-KDS-05
SS-05	Events for one ticket delivered shuffled and duplicated into the client merge function	The final state equals the snapshot (a property test over many random orderings)	API 12.2 rule 2
SS-06	A consumer that reads nothing; 150 events published	The server closes that stream; the overflow counter becomes 1; other subscribers are unaffected; a reconnect works	SAD 9.4
SS-07	Token life of 2 seconds	STREAM_CLOSING with reason TOKEN_EXPIRED, then the stream closes within 1 second of expiry; a reconnect with a fresh token works	SAD 9.6
SS-08	The 4th stream of one user; the 21st from one address; the 51st overall	429 RATE_LIMITED with rateClass CONNECTION_CAP each time; closing a stream frees a slot	SAD 5.2

ID	Case	Expected	Proves
SS-09	Tickets for Grill and Fry; streams for Fry, Grill and Manager	Fry never receives a Grill event; Manager receives both	FR-AUTH-04
SS-10	Unbound Kitchen user; expired token; unknown station; Accept of application/json	403; 401 before any stream starts; 404; 406	RM-02
SS-11	A payment transaction forced to roll back; a committed one	The rolled-back one publishes nothing; the committed one publishes exactly once per ticket	SAD P4
SS-12	The application is stopped gracefully during an open stream	SNAPSHOT_REQUIRED with reason SERVER_SHUTDOWN, then close; an in-flight request still completes within the 20-second grace period	SAD 11.2
SS-13	The event executor blocked and its queue filled	SNAPSHOT_REQUIRED with reason OVERLOAD reaches every subscriber, so no event is lost silently	LLD 3.11
SS-14	The public board stream with no token	BOARD_CHANGED on READY, recall, complete and cancel; the payload holds only tokens and times; the polling endpoint returns the same state	FR-BRD-01 to 04
SS-15	A real Nginx container in front	Events arrive within 1 second (no buffering); a stream stays open for 90 seconds, beyond the default proxy timeouts; no compression on streams	SAD 9.7
SS-16	The manager stream	LATE_TICKET once per ticket when it crosses its red threshold; ORDER_CHANGED on status changes; CONFIG_CHANGED on configuration	FR-KDS-09

ID	Case	Expected	Proves
SS-17	Client reconnect logic (Vitest)	Delays 1, 2, 4, 8, then 15 seconds at most, each with jitter	API 12.2 rule 4
SS-18	Client dead-connection detection (Vitest)	No heartbeat for 40 seconds triggers a reconnect	API 12.2 rule 3
SS-19	Client fallback (Vitest)	While the stream is down a station polls its snapshot every 2 seconds and the board every 3	API 12.2 rule 6

9. Rate-limit and cache tests

9.1 Rate limiting (RL)

Buckets take an injected time meter, so refill is tested instantly. Limits are the defaults of API 2.10 unless a case says otherwise.

ID	Case	Expected	Proves
RL-01	For each class (LOGIN, REFRESH, ORDER_WRITE, TICKET_ACTION, READ, REPORT, ADMIN_WRITE, BOARD_READ, SSE_CONNECT): with time frozen, send as many requests as the burst; send one more; advance by one refill period	The burst all pass; the next gets 429 RATE_LIMITED with details.rateClass and Retry-After of at least 1; after the refill the burst passes again. X-RateLimit-Limit equals the capacity and X-RateLimit-Remaining falls by one per call	SAD 5.2, API 2.10
RL-02	User A exhausts ORDER_WRITE	User B is unaffected; A can still use TICKET_ACTION and READ (classes are independent)	Per-user, per-class keys
RL-03	ORDER_WRITE exhausted (30 per minute)	Retry-After equals the seconds until the next token, rounded up, at least 1	LLD 3.7

ID	Case	Expected	Proves
RL-04	Five wrong passwords, then the correct one; then 15 minutes later (clock) the correct one	The fifth is 401; the sixth is 429 with Retry-After of about 900 seconds; later 200; a successful login resets the counter	FR-AUTH-06
RL-05	Six attempts with an unknown username, and six with a known user locked out	Both give 429 with the same envelope shape and headers: lockout reveals no usernames	SC-22
RL-06	The same username from another address; a different username from the same address	The bucket is keyed on both, so the second pair is not blocked by the first; the database lockout still applies to the real user	LLD 3.7
RL-07	The bucket store throws an unexpected error	Normal requests succeed, an ERROR is logged and kf_rate_limiter_errors_total rises; a login attempt fails closed with 503 SERVICE_BUSY	SAD 5.2, P5
RL-08	1,000 health calls; static files; stream heartbeats	None is limited	SAD 5.2
RL-09	20,000 requests with distinct keys; then 10 idle minutes (clock)	The bucket cache never exceeds 10,000 entries; idle entries expire	LLD 3.7
RL-10	A request from outside the trusted proxy carrying X-Forwarded-For; the same from the Nginx container	The header is ignored in the first case and honoured in the second, so a client cannot forge its address	SAD 5.2
RL-11	k6 at the design workload (150 orders an hour with the screens' normal reads) for 10 minutes, with real per-user limits	Zero 429 responses	NFR-SEC-06 (proposed)

ID	Case	Expected	Proves
RL-12	Three boards behind one address polling every 2 seconds, then every 3 seconds	Every 2 seconds: 429 appears within a minute; every 3 seconds: no 429 over 2 minutes	API 2.10
RL-13	41 requests to /api from one address within a second, through real Nginx	The surplus gets 429 from Nginx with a JSON body that carries code RATE_LIMITED	SAD 5.2 edge layer
RL-14	The sixth login request within a minute from one address, through real Nginx	Blocked at the edge	SAD 5.2 edge layer

9.2 Caching (CA)

ID	Case	Expected	Proves
CA-01	Two GET /menu with the same filter; a different filter; more than 64 distinct filters	The loader runs once for the first pair; a different filter is a separate entry; the cache never holds more than 64 entries	SAD 5.3
CA-02	Change an item through the API; then another change that is rolled back	The next GET /menu shows the change with a new ETag within 1 second; the rolled-back change neither evicts nor changes the ETag	FR-MENU-04, FR-INV-03
CA-03	The loader is held after reading old data; a change commits and eviction is requested; the loader is released	The next GET /menu returns the new data: no stale entry survives (50 repeats)	SAD 5.3 race rule
CA-04	If-None-Match equal to the current ETag; a different value; a different filter	304 with no body; 200; a different ETag	API 2.9

ID	Case	Expected	Proves
CA-05	Change recall_window_seconds, then recall	The recall uses the new value at once. With eviction disabled (mutation) the new value appears only after the 60-second backstop	FR-CFG-01
CA-06	100 board requests in one second	One database query; a change appears within 1 second	LLD 3.8
CA-07	Warm the menu cache; change a price directly in SQL, bypassing eviction; create an order	The order is priced at the new database price while GET /menu still shows the old one: the database decides, the cache only displays	SAD P1
CA-08	Warm the cache with an item available; mark it unavailable directly in SQL; create an order	422 ITEM_UNAVAILABLE although the cache says available	BR-04
CA-09	Advance the cache ticker by 5 minutes	The menu entry is reloaded	SAD 5.3 backstop
CA-10	The cache throws on load	GET /menu still returns 200, loaded straight from the database	SAD 11.4
CA-11	100 menu requests after warm-up	Hit ratio above 95 percent in the exported metrics	SAD 5.3

10. Security tests (SC)

ID	Case	Expected
SC-01	A token with the role claim edited but not re-signed; algorithm none; another signing key; expired; no expiry	401 for each
SC-02	Station binding and state-dependent roles	As RM-02 and RM-03

ID	Case	Expected
SC-03	Price, total, status, role, active or password hash sent in a create or patch body	400 MALFORMED_REQUEST (unknown property)
SC-04	SQL metacharacters (a quote followed by OR 1=1, a comment marker) in note, search q, username, reference and audit action	Stored or matched as plain text; no SQL error; no table touched
SC-05	A JSON document nested 5,000 levels deep	400, no stack overflow
SC-06	A form-encoded body to a JSON endpoint	415
SC-07	CORS: an allowed origin; another origin; credentials	Allowed origin gets the exact origin back (never a wildcard with credentials); another origin gets no CORS headers; credentials only on the auth endpoints
SC-08	Security headers on API and static responses	Content-Security-Policy with default-src 'self', X-Content-Type-Options nosniff, Referrer-Policy no-referrer, frame-ancestors none; HSTS on the HTTPS profile
SC-09	Capture the logs through login, refresh, a failed login and a password change	No password, token, Authorization header or refresh cookie appears
SC-10	Force an unexpected exception and a database constraint failure	The body is the generic envelope: no class name, SQL text or stack trace
SC-11	200 logins for an unknown user and 200 with a wrong password for a known user	Median response times within 25 percent of each other (a statistical check, run nightly)
SC-12	Refresh cookie attributes	HttpOnly, Secure, SameSite=Strict, path /api/v1/auth, Max-Age of 8 hours
SC-13	Refresh with a foreign Origin, and with no Origin	403 each

ID	Case	Expected
SC-14	Try to change or delete audit rows through the API and through SQL as kf_app	No endpoint exists; the database refuses (DB-06, DB-07)
SC-15	Secret scanner over the repository, and a deliberate fake key committed on a scratch branch	Clean on the main branch; the scratch branch fails the build
SC-16	Dependency vulnerability scan	No High or Critical finding, or each exception documented
SC-17	Add a controller method with no URL rule on a scratch branch	AR-06 fails the build (deny by default holds)
SC-18	Brute-force login	RL-04 and RL-05
SC-19	A note containing a script tag	Returned unchanged as a JSON string with nosniff; rendered as plain text on the station and manager screens (Playwright)
SC-20	A request with only the refresh cookie and no Authorization	401 on bearer endpoints
SC-21	A username or correlation id containing a newline	Never written raw into a log line
SC-22	Login, lockout and refresh failures for existing and non-existing users	Identical messages and envelopes
SC-23	Stored password hash	BCrypt with cost 12; verification takes between 100 and 600 ms on the reference machine
SC-24	Access token lifetime	exp minus iat equals 900 seconds
SC-25	A user demoted after login	The new token carries the new role; the old token keeps the old role until it expires (documented behaviour)

11. Resilience and fault tests (RS)

ID	Fault	Expected	Proves
RS-01	Ticket creation fails after the payment row is inserted (a failing TicketGateway)	500 INTERNAL_ERROR; the order is still CREATED at its old version; no payment, no tickets, no audit, no events; after the fault is removed the retry succeeds	AT-11, FR-PAY-02, NFR-REL-01
RS-02	The audit insert fails during create, pay, bump and cancel	Each business change is rolled back completely	FR-AUD-04
RS-03	The database is stopped, then restarted	Readiness is DOWN and /health returns 503; API calls return 503 SERVICE_BUSY with Retry-After within about 2 seconds; open streams keep their heartbeat; within 30 seconds of the restart the service recovers without restarting the application	SAD 11.1
RS-04	All 20 pool connections held by the test	The next request returns 503 SERVICE_BUSY with reason POOL after about 2 seconds; normal service resumes when they are released	SAD 5.4
RS-05	Another session holds the order row for 3 seconds	A bump returns 503 SERVICE_BUSY with reason LOCK_TIMEOUT at about 2 seconds, and is not retried	SAD 6.5
RS-06	A statement made to run too long: 11 seconds in a report, 6 seconds in a normal call	503 SERVICE_BUSY with reason QUERY_TIMEOUT in both	SAD 5.4
RS-07	The application is killed while 20 orders a second are being created, then restarted	Committed orders are intact; no partial rows; C1 to C7 clean; screens reconnect and match the snapshot within 3 seconds of the restart	NFR-REL-02

ID	Fault	Expected	Proves
RS-08	Graceful stop with transactions in flight	All of them complete; none is half-done; streams receive SNAPSHOT_REQUIRED	SAD 11.2
RS-09	The event executor is saturated	OVERLOAD resynchronisation reaches every subscriber	SS-13
RS-10	The rate limiter breaks	Fail open, except login	RL-07
RS-11	The cache breaks	Database fallback	CA-10
RS-12	A scheduled job throws	The error is logged and the next run still happens	LLD 3.13
RS-13	Client clock 10 minutes fast and 10 minutes slow	Station timers and colours equal those of a correct clock, because they use server timestamps (UI test with a mocked Date)	API 15.4
RS-14	Duplicate and reordered stream events	Harmless	SS-05
RS-15	A deliberately broken migration in a copy of V1	The application exits at start-up with a non-zero code; readiness never turns UP; the database is unchanged	NFR-REL-03
RS-16	PostgreSQL restarted while the application runs	The pool reconnects; the first requests may return 503; the application recovers without a restart	SAD 11.1
RS-17	1,000 stream connect-and-drop cycles, including sockets reset in the middle of an event	The open-stream gauge returns to 0 and the thread count returns to its starting value	SAD 9.4

12. Performance and load tests (PF)

12.1 Method

Performance runs use the reference machine (4 cores, 8 GB, application and database on it), the loadtest profile (raised per-user limits, production timeouts and pool) and the 100,000-order dataset. Each scenario warms up for 60 seconds, measures for 2 minutes at steady load, and is run 3 times; the median run is reported. k6 provides client-side numbers; Micrometer and pg_stat_statements provide server-side numbers. Results are saved, and a regression of more than 20 percent against the previous run is investigated before release.

12.2 Scenarios

ID	Scenario	Load	Pass criteria	Source
PF-01	Order creation	50 concurrent users create orders of 1 to 3 lines from untracked items, each with a unique Idempotency-Key	Creation p95 under 300 ms; failures under 0.1 percent; pending pool connections stay at 0	NFR-PERF-01
PF-02	Kitchen push latency	10 open streams; 5 payments a second for 2 minutes (600 payments)	Event delivery p95 at most 1 second; every ticket event received exactly once per interested stream	NFR-PERF-02
PF-03	Order lists and search	20 users run random filters (status, day, token, order number), pages of 20, plus cursor mode	p95 under 300 ms	NFR-PERF-03
PF-04	Reports	One request at a time over a 31-day range: sales, top items, prep times, hourly	Each p95 under 2 seconds	NFR-PERF-04

ID	Scenario	Load	Pass criteria	Source
PF-05	Audit search	20 users search by entity and by date over 500,000 rows	p95 under 300 ms	FR-AUD-03
PF-06	Design-peak soak	150 orders an hour, normal reads, 7 streams, for 4 hours	Zero errors; heap, thread count and open-stream gauge flat after warm-up (within 10 percent); creation p95 under 150 ms	SRS 2.5
PF-07	Stress	Ramp from 50 to 300 users over 10 minutes creating orders	Latency rises but the only errors are 429 and 503 SERVICE_BUSY: no 500 and no crash; the service recovers within 30 seconds after the load stops; C1 to C7 clean	SAD 5.4
PF-08	Spike	100 orders in 10 seconds from 20 users	All succeed; tokens unique; p95 under 500 ms	SAD 2.3
PF-09	Report bulkhead	During PF-01, five concurrent report requests repeatedly	At most 2 run at once; the others get 429 with rateClass REPORT_CONCURRENCY; order creation p95 rises by at most 10 percent	SAD 5.4
PF-10	Pool and timeouts	During PF-01	No connection timeouts; maximum active connections at most 20	SAD 5.4
PF-11	Start-up	docker compose up to readiness	Under 60 seconds on an empty database; under 90 seconds with the 100,000-order dataset	NFR-POR-01
PF-12	Data scripts	Seed and the 100,000-order generator	Under 60 seconds and under 2 minutes	FR-CFG-02, FR-CFG-03
PF-13	Menu reads	200 users request /menu with their ETags	p95 under 50 ms; cache hit ratio above 95 percent	FR-MENU-06

ID	Scenario	Load	Pass criteria	Source
PF-14	Board reads	50 users request /board continuously	p95 under 50 ms; at most about one database query per second	LLD 3.8
PF-15	Stream capacity	50 streams (the cap) and 20 events a second	No lost events; sender threads under 50 percent busy; memory per stream recorded	SAD 9.4
PF-16	Login cost	20 logins a second for 1 minute	No failures other than 429 from the limiter; CPU use recorded, as BCrypt at cost 12 is deliberately expensive	SC-23

12.3 k6 outline for PF-01

```
import http from 'k6/http';
import { check } from 'k6';
import { uuidv4 } from 'https://jslib.k6.io/k6-utils/1.4.0/index.js';

const BASE = __ENV.BASE_URL || 'http://localhost:8080/api/v1';

export const options = {
  scenarios: {
    create_orders: {
      executor: 'ramping-vus',
      stages: [ { duration: '30s', target: 50 }, { duration: '2m', target: 50
}, { duration: '10s', target: 0 } ],
    },
  },
  thresholds: {
    'http_req_duration{endpoint:create_order}': ['p(95)<300'],
    'http_req_failed{endpoint:create_order}': ['rate<0.001'],
  },
};

export function setup() {
  // log in the 10 seeded cashier accounts used by the load profile and
return their tokens
  return loginCashiers();
}
```

```
export default function (tokens) {
  const token = tokens[__VU % tokens.length];
  const res = http.post(`${BASE}/orders`, JSON.stringify({ orderType:
'TAKEAWAY', lines: pickLines() }), {
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer
${token}`, 'Idempotency-Key': uuidv4() },
    tags: { endpoint: 'create_order' },
  });
  check(res, { created: (r) => r.status === 201 });
}
```

12.4 Review after each run

Rank pg_stat_statements by total time and confirm that no hot-path statement averages above 100 ms; confirm that the plans still match DBD 7.2; save the k6 summary, the Micrometer export and the plans next to the run's date.

13. Acceptance scenarios (AT)

These are the 11 scenarios of SRS section 12.2 as end-to-end scripts. They run against the Compose stack with the seed data (DBD 10), either from the screens or from the Bruno collection. Common set-up: the stack is up, the database freshly seeded, and the users cashier1, manager1, admin1, fry1, grill1, assembly1, beverage1 and multi1 exist.

AT-01 Happy path.

1. Log in as cashier1, fry1, assembly1 and beverage1. Open the streams of the Fry, Assembly and Beverage stations and the board stream.
2. GET /menu and note the ETag.
3. As cashier1, POST /orders (TAKEAWAY): 2 Spicy Chicken Burger with Hot and Extra cheese, 1 Cola with Large, with key K1. Expect 201, status CREATED and total 513.72.
4. GET the bill: provisional, paid false, total 513.72.
5. POST a UPI payment of 513.72. Expect 201 and an order that is IN_KITCHEN with three tickets.
6. Within 1 second each station stream shows its ticket (the burger on Fry and on Assembly, the cola on Beverage), and the board lists the token as preparing.
7. fry1, assembly1 and beverage1 each start and bump their ticket. The third bump reports becameReady true.
8. The board moves the token to ready; the stations show no tickets.

9. cashier1 completes the order. Expect COMPLETED and the token gone from the board.
10. As manager1, read the audit trail of the order.

Pass when every expectation holds, the audit shows ORDER_CREATED, ORDER_PAID, ORDER_IN_KITCHEN, three TICKET_STARTED, three TICKET_BUMPED, ORDER_READY and ORDER_COMPLETED (and PAYMENT_RECORDED for the payment), the bill totals still read 513.72, and C1 to C7 are clean.

AT-02 Validation. As manager1, mark Cola unavailable (M-17). As cashier1, POST an order with Cola quantity 1 and Classic Veg Burger quantity 0. Expect one 422 VALIDATION_FAILED listing lines[0].itemId with ITEM_UNAVAILABLE and lines[1].quantity with OUT_OF_RANGE; no order created; token counter and stock unchanged. Then POST an order containing only the unavailable Cola: expect top-level 422 ITEM_UNAVAILABLE.

AT-03 Idempotent retry. Send the same POST /orders with one key from 50 threads at once. Expect one order, 50 responses of 201 with identical bodies, 49 with Idempotency-Replayed true. Then reuse the key with a different payload: 422 IDEMPOTENCY_KEY_REUSED. Finally simulate a lost response (cut the connection after the server committed) and retry: the original order comes back.

AT-04 Last item. Choco Lava Cake has stock 10. Send 100 concurrent orders of one unit each. Expect exactly 10 created and 90 failing with 409 OUT_OF_STOCK; stock 0; GET /menu shows the item unavailable. As manager1 add 5 units (I-02): the item is available again and one more order succeeds.

AT-05 Concurrent ticket update. With an IN_PROGRESS ticket at Fry, fry1 and multi1 bump it at the same moment. Expect one 200 and one 409; the ticket is DONE once; one TICKET_BUMPED audit record.

AT-06 Recall. Bump the last ticket of an order: it becomes READY and the board shows it ready. Within 10 minutes, fry1 recalls the ticket: the ticket is IN_PROGRESS, the order IN_KITCHEN and the token is back in preparing. Bump again for READY. Advance the clock 11 minutes and recall: 409 with reason RECALL_WINDOW_EXPIRED. Complete the order and recall: 409 with reason ORDER_COMPLETED.

AT-07 Cancellation rules. (a) cashier1 cancels a CREATED order: 200, stock restored. (b) cashier1 cancels an IN_KITCHEN order: 403. (c) manager1 cancels that order with a reason: 200, tickets CANCELLED with notices on the stations, stock not restored, a pending refund. (d) manager1 cancels a READY order: 409. (e) Cancelling the cancelled order again: 409 with currentStatus CANCELLED.

AT-08 Access control. Run the role matrix RM-01 and the station cases RM-02, RM-03 and RM-04. Expect 401 or 403 exactly as specified every time, and a deactivated user unable to log in.

AT-09 Reconnect. Open a station stream, then cut it (stop the proxy) for 30 seconds while 3 orders are paid. Restore it. Expect the client to reconnect with its backoff, reload the snapshot, and within 3 seconds show all open tickets in the right order with no duplicate.

AT-10 Reports. Load the known report dataset and run R-01 to R-05: every figure equals the stored expected sheet. Run the four main reports on the 100,000-order dataset: each p95 under 2 seconds.

AT-11 Fault injection. Switch on the test fault that makes ticket creation fail, and pay a CREATED order. Expect 500; the order still CREATED at its old version; no payment, no tickets, no audit, no events. Switch the fault off and pay again: 201.

14. Usability checks and the demo

14.1 Usability and compatibility (US)

ID	Check	Method	Pass criteria
US-01	A newcomer enters a 3-item order and records payment	3 testers, a 5-minute walkthrough, a stopwatch	Each finishes within 60 seconds (NFR-USE-02)
US-02	Readability of the station and board screens	Checklist and automated measurement	Item names at least 24 px; status shown by text or icon as well as colour (a greyscale screenshot still readable); touch targets at least 44 px; legible from 1.5 metres (NFR-USE-01)
US-03	Browsers and devices	AT-01 run on current Chrome, Edge and Firefox on a desktop, and Chrome at 10-inch landscape tablet size	Every step completes; streams work; no console errors

ID	Check	Method	Pass criteria
US-04	Contrast and keyboard	An automated accessibility scan; keyboard-only use of the admin screens	No contrast failures at the AA level; every admin action reachable
US-05	Error messages on the POS	Trigger each error below and read the screen	The behaviour in the table

Error	Expected screen behaviour
OUT_OF_STOCK	Names the short items and offers to remove them
ITEM_UNAVAILABLE	Greys out the item and asks to remove it
AMOUNT_MISMATCH	Says the order changed and reloads the current total
STALE_VERSION	Reloads the current state with a short notice
RATE_LIMITED	Shows how many seconds to wait
SERVICE_BUSY	Shows a retry banner and retries with backoff
UNAUTHENTICATED	Refreshes the token silently and repeats the call; shows the login screen if that fails
FORBIDDEN	Says the action is not allowed for this user
Network offline	Shows an offline banner; station screens keep polling

14.2 The demo (DM)

DM-01 The 5-minute demo script

Time	Action	Expected	What to say
0:00	Show the running stack, the seeded menu, the station screens and the board	Everything is up	The system takes an order, sends it to the right stations and tracks it to the counter

Time	Action	Expected	What to say
0:30	Create and pay the 513.72 order	Three tickets appear on three screens in under a second	Pay first, then the order is routed by station
1:30	Start and bump the tickets; complete the order	The board shows READY, then the token leaves	The order becomes READY exactly once, even if two cooks bump together
2:30	Run the concurrency script: 100 orders for the item with stock 10; then 50 identical retries with one key	10 orders and 90 clean OUT_OF_STOCK; one order for the 50 retries	Conditional update for stock, insert-first idempotency
3:15	Stop the proxy for 20 seconds, then restore it	Screens reconnect and match the queue	The database is the truth; the push is only a hint
3:45	As the manager, show the reports and the audit trail of the order	Figures and a complete trail	Audit is written in the same transaction
4:30	Show CI green and the stuck-order mutation test	Tests pass; the mutation test shows the bug without the lock	How I proved it

DM-02 Before the demo and fallbacks

Check or risk	Action
Clean start	Database reset and seeded, all services healthy, clock and time zone correct
Accounts	Passwords known; one token already refreshed
Rehearsal	The script run twice on the demo machine
Stream demo fails	Show the polling fallback and explain the resynchronisation

Check or risk	Action
Machine fails	A recorded video of the full script and the exported test reports
A question goes deep	Open the architecture decision records (SAD section 14) and the concurrency section (SAD section 6)

15. Traceability matrix

Every test method carries a @Covers annotation naming the requirement IDs it proves. A check, AR-14, reads the annotations and fails the build when any Must requirement has no test, so this matrix cannot go stale silently. CT followed by an endpoint ID means the contract cases of that endpoint (section 6.1).

15.1 Functional requirements

Requirement	Tests
FR-AUTH-01	CT A-01, SC-01, SC-23, SC-24, RM-04, AT-08
FR-AUTH-02	SC-09, SC-23, PC-11
FR-AUTH-03	RM-01, AR-06, SC-17, AT-08
FR-AUTH-04	RM-02, SS-09, SS-10, CT K-01 to K-06
FR-AUTH-05	CT U-01 to U-05, PC-11, CC-12, RM-05
FR-AUTH-06	CT A-01, RL-04, RL-05, RL-06, CC-14
FR-AUTH-07	CT A-02, CT A-03, CC-13, SC-12, SC-13
FR-MENU-01 to 03	CT M-02 to M-16, DB-04, PC-10
FR-MENU-04	CT M-17, PC-08, CC-11, CA-02, CA-08, AT-02
FR-MENU-05	CT M-09, DB-05, PC-10
FR-MENU-06	CT M-01, WS-08, CA-01 to CA-04, CA-11, PF-13
FR-MENU-07	CT M-18 to M-21, UT-16, PC-04
FR-MENU-08	UT-02, PC-01, CA-07

Requirement	Tests
FR-ORD-01, FR-ORD-02	CT O-01, UT-10, WS-02, AT-02
FR-ORD-03	UT-01, UT-02, DB-04, CT O-01, PC-01
FR-ORD-04	UT-14, CC-08, DB-13
FR-ORD-05	UT-06, CT O-04 to O-06, CC-06
FR-ORD-06	CT O-04, PC-02, CC-07
FR-ORD-07	CT O-05, PC-03, RM-03, AT-07
FR-ORD-08	CT O-06, AT-01
FR-ORD-09	CT O-02, WS-07, DB-09, PF-03
FR-ORD-10	CT-ID-01 to CT-ID-08, CC-01, CC-01m, AT-03
FR-ORD-11	CT O-07, DB-10 (C5)
FR-ORD-12	PC-14, CC-09
FR-PAY-01, FR-PAY-02	CT P-01, PC-04, RS-01, AT-01, AT-11
FR-PAY-03	CC-05, CC-05m, DB-03, CT P-01
FR-PAY-04	CT P-01, CC-07
FR-PAY-05	CT P-03, UT-02, PC-04
FR-PAY-06	CT P-01, DB-04
FR-PAY-07	CT P-05, CT P-06, PC-03, PC-13
FR-PAY-08	CT P-04
FR-KDS-01	UT-16, PC-04
FR-KDS-02	CT K-01, DB-09, DB-15, UT-08
FR-KDS-03	CT K-03, CT K-04, PC-05, PC-06, AT-01
FR-KDS-04	CT K-05, PC-07, AT-06
FR-KDS-05	SS-01 to SS-08, SS-11 to SS-13, SS-15, SS-17 to SS-19, AT-09, PF-02
FR-KDS-06	CC-04, CC-04m, PC-06, AT-06

Requirement	Tests
FR-KDS-07	CC-03, CT-VER-01, AT-05
FR-KDS-08	CT K-06, PC-03, DB-04, AT-07
FR-KDS-09	CT K-07, SS-16
FR-KDS-10	CT K-01, DB-15, US-02
FR-BRD-01 to 04	CT B-01, SS-14, PC-04, PC-06, PC-07, PF-14, US-02
FR-INV-01 to 05	CT I-01 to I-03, CC-02, CC-02m, CC-15, PC-09, AT-04
FR-RPT-01 to 07	CT R-01 to R-05, DB-16, AT-10, PF-04, PF-09
FR-AUD-01	CT AU-01, PC-01 to PC-14, AT-01
FR-AUD-02	DB-06, DB-07, SC-14
FR-AUD-03	CT AU-01, PF-05
FR-AUD-04	RS-02
FR-CFG-01	CT C-01, CT C-02, PC-12, CA-05
FR-CFG-02, FR-CFG-03	DB-11, DB-12, PF-12
FR-SYS-01	WS-01, section 6.1 envelope checks
FR-SYS-02	AR-08
FR-SYS-03	WS-07
FR-SYS-04	WS-05, SC-21
FR-SYS-05	CT H-01, CT H-02, RS-03

15.2 Business rules and quality requirements

Requirement	Tests
BR-01	CC-08, DB-13, UT-14
BR-02, BR-03	UT-01, UT-02, DB-04
BR-04	CT O-01, CA-08, CC-11

Requirement	Tests
BR-05	UT-06, CT O-05, PC-03, RM-03
BR-06	CC-04
BR-07, BR-08	CT P-01, DB-04, CC-07
BR-09	DB-12 (C6), UT-16, CT M-06, CT M-10
BR-10	DB-15, CT K-01, UT-16
BR-11	UT-08, CT K-01
BR-12	CC-02, CC-15, PC-03, DB-04
BR-13	PC-01 to PC-14, RS-02
BR-14	CT O-04
BR-15	CT-ID-01 to CT-ID-08, CC-01
BR-16	UT-05, PC-01
BR-17	CT K-05, AT-06
BR-18	UT-04, DB-14
NFR-PERF-01, 03, 04	PF-01, PF-10, PF-03, PF-04
NFR-PERF-02	SS-02, PF-02
NFR-CONC-01 to 04	CC-01, CC-02, CC-03, CC-04 and their variants
NFR-SEC-01	SC-01, SC-15, SC-24
NFR-SEC-02	WS-02, SC-03, SC-04
NFR-SEC-03	SC-07, SC-08
NFR-SEC-04	SC-09 and a schema review showing no customer personal data
NFR-SEC-05	SC-01 to SC-25 as the evidence for the OWASP Top 10 review
NFR-REL-01	RS-01, RS-02, DB-05
NFR-REL-02	RS-07, SS-04

Requirement	Tests
NFR-REL-03	DB-01, RS-15
NFR-USE-01, 02	US-02, US-01
NFR-MNT-01 to 03	AR-01 to AR-05, AR-13
NFR-OBS-01, 02	WS-05, SC-21, CA-11, RL-07
NFR-POR-01, NFR-DOC-01	PF-11, DM-01, AR-08
Goal G1 (correctness under rush)	CC-01 to CC-15, DB-10, AT-03 to AT-06
Goal G2 (real-time visibility)	SS-01 to SS-19, PF-02, AT-09
Goal G3 (lifecycle and audit)	UT-06, UT-07, PC-01 to PC-14, AT-01, AT-07
Goal G4 (kitchen performance)	CT R-01 to R-05, DB-16, AT-10
Goal G5 (easy to run and show)	DB-11, DB-12, PF-11, DM-01
Goal G6 (defensible engineering)	AR-01 to AR-14, SC-01 to SC-25, the mutation variants, the CI gates

16. Schedule, risks and open items

16.1 Tests to write at each build step

The steps are those of LLD section 7. Write these tests first at each step.

Step	Tests to write first
1	DB-01, DB-02
2	UT-01 to UT-05, UT-14
3	WS-01 to WS-05, WS-09, WS-10
4	RM-01 (for implemented endpoints), RM-04, AR-06, SC-01, SC-17
5	CT A-01 to A-05, CT U-01 to U-04, UT-09, RL-04 to RL-06, CC-14, SC-11 to SC-13, SC-22 to SC-25
6	DB-12

Step	Tests to write first
7	CT M-01 to M-17, WS-08, CA-01 to CA-04, CA-09 to CA-11
8	UT-02, UT-10, CT O-01 to O-03, PC-01, WS-07
9	UT-06, CT O-04 to O-07, PC-02, PC-03, PC-14, RM-03, CC-06, CC-07, CC-09
10	UT-16, CT P-01 to P-06, PC-04, RS-01, AT-11, CC-05, CC-05m
11	UT-07, UT-08, CT K-01 to K-07, PC-05 to PC-07, CC-03, CC-04, CC-04m, RM-02, AT-05, AT-06
12	SS-01 to SS-19, CT B-01, RS-09
13	CT-ID-01 to CT-ID-08, CC-01, CC-01m, AT-03
14	CT I-01 to I-03, CC-02, CC-02m, CC-15, PC-09, AT-04
15	CT AU-01, RS-02, SC-14
16	RL-01 to RL-14, CA-05 to CA-08, CC-10, CC-10b, RS-04 to RS-06, RS-10 to RS-12
17	The full concurrency suite repeated 50 times, with every mutation variant
18	CT R-01 to R-05, DB-16, AT-10, PF-09
19	PF-01 to PF-16, DB-09, DB-11, AR-07 to AR-14, RS-03, RS-07, RS-08, RS-13 to RS-17, SS-15, AT-08, US-01 to US-05, DM-01

16.2 If time runs short

Cut in this order, and never cut the first two groups: (1) the concurrency suite with its mutation variants and the acceptance scenarios AT-01 to AT-11; (2) the contract cases and post-conditions of Must endpoints; then, from the bottom up: usability checks, the statistical and nightly security cases (SC-11, SC-25), the Could and Should endpoint cases, stress and soak runs (PF-06, PF-07, PF-15), and the Nginx proxy cases (SS-15, RL-13, RL-14).

16.3 Risks to the test effort

Risk	Mitigation
The volume of cases is large	One generic runner reads the YAML files; adding a case is adding data, not code
Concurrency tests are flaky	Barriers and hooks make the interleavings deterministic where it matters; the failure budget is zero, so a flake is investigated at once
Performance numbers depend on the machine	One named reference machine; thresholds copied from the SRS; three runs and the median
Production code gains test seams	The TestHooks beans do nothing in production and exist only for the mutation variants (open item OI-05)
The API document and the YAML drift apart	AR-07 compares them on every build

16.4 Open items

ID	Item	Decide in
OI-01	Automate the screen checks (SC-19, US-02, RS-13) with Playwright, or run them as manual checks	Phase 4
OI-02	The performance thresholds are set for the reference machine; record the machine used for the demo	Phase 4
OI-03	The timing check SC-11 is statistical; accept a nightly-only status if it proves noisy	Phase 3
OI-04	Generate the case YAML from the exported API document with a script, or write it by hand	Phase 1
OI-05	Confirm that no-op test seams (TestHooks) in production code are acceptable	Phase 2

ID	Item	Decide in
OI-06	Run contract cases in parallel to shorten the run, or keep them sequential	Phase 3

17. Consolidated change register

Documents 3 to 6 each proposed changes to earlier documents. They are gathered here, de-duplicated, as one checklist. Apply them in one pass before the code reaches the steps they affect, producing SRS v0.2, SAD v0.2, DBD v0.2, API v0.2 and LLD v0.2, and tick each one off.

#	Target	Change	Origin
1	SRS	Add error codes METHOD_NOT_ALLOWED, NOT_ACCEPTABLE, RESOURCE_IN_USE, PAYLOAD_TOO_LARGE, UNSUPPORTED_MEDIA_TYPE and SERVICE_BUSY. Define INVALID_TRANSITION as an operation not allowed in the current state. State the ITEM_UNAVAILABLE versus VALIDATION_FAILED rule	SAD 15.2, API 17.2
2	SRS	Add as Should: change-password, admin password reset, refund completion and the stock list. FR-AUTH-06: lockout state is persisted, and unknown usernames are throttled like known ones	SAD 15.2, API 17.2
3	SRS	Add the limits (30 lines per order, 10 modifier groups per item, 20 modifiers per group, prices up to 99999.99, passwords of 8 to 72 bytes) and two API rules: unknown properties and parameters are rejected, and money is carried as strings	API 17.2
4	SRS	Add NFR-SEC-06 (rate limits above the design workload, no 429 at that workload) and NFR-PERF-05 (reports limited to 2 concurrent queries)	SAD 15.2

#	Target	Change	Origin
5	SRS	Update FR-KDS-05 (SNAPSHOT_REQUIRED and STREAM_CLOSING events; streams close at token expiry), FR-MENU-06 (ETag and 304), FR-ORD-09 (cursor mode, 92-day cap) and FR-RPT-03 (the per-item measure is the cycle time of the item's ticket)	SAD 15.2, API 17.2
6	SRS	Section 9: add menu_version and order_line_component; ticket.business_day, rush, cancel_ack_at, recall_count; orders.cancel_reason_code, cancel_reason_text, cancelled_by; payment.change_due (and FR-PAY-06 stores it); IdempotencyRecord without a status; the partial indexes in DR-07; record OI-02 and OI-03 as closed	DBD 15.2, SAD 15.2
7	SAD	Section 4.1: station is owned by config. Section 5.3: the menu cache holds one entry per filter combination (at most 64), and the board snapshot is cached for 1 second	DBD 15.2, LLD 8.2
8	SAD	Section 5.2 and 9: add the REPORT_CONCURRENCY and CONNECTION_CAP details, the guidance for polling boards, the STREAM_CLOSING event, and SNAPSHOT_REQUIRED with reasons SERVER_SHUTDOWN and OVERLOAD	API 17.2, LLD 8.2
9	SAD	DD-02: batch inserts need sequence ids with INCREMENT 50. Section 5.4: the report statement timeout is set per transaction with SET LOCAL. Section 16.3: replace the state machine sketch with the two-step check (no rule gives 409, an excluded role gives 403)	DBD 15.2, LLD 8.2

#	Target	Change	Origin
10	DBD	Grant kf_app DELETE on menu_item, modifier_group, modifier, item_stock and combo (referenced rows stay protected by foreign keys), and on order_line, order_line_modifier and order_line_component with the guard trigger of LLD 8.2; amend DD-10 to allow the guard	API 17.2, LLD 8.2
11	DBD	S5 gains AND total = :amount; S3 returns the new quantity; add S12b (per-item cycle time) and S14 (cancellations by reason and role), whose SQL is in LLD 8.2	API 17.2, LLD 8.2
12	API	Section 2.5: authentication (401) comes before the syntax checks, and the URL-level 403 before body parsing; renumber the table	LLD 8.2
13	API	Section 12.2: SNAPSHOT_REQUIRED carries reason SERVER_SHUTDOWN or OVERLOAD	LLD 8.2
14	LLD	Section 2.3: make the stream heartbeat (default 15 seconds) and the emitter margin configuration properties, so tests can shorten them	This document, 3.4
15	LLD	Section 3.4: set spring.jackson.parser.strict-duplicate-detection to true, so a duplicate JSON property is a 400	This document, WS-02
16	LLD	Section 4.6: SlaCalculator compares exact products (age times 100 against the threshold times the target), not a truncated percent; the 301-second vector of UT-08 guards it	This document, UT-08
17	LLD	The Nginx configuration answers its own 429 and 413 responses with the standard JSON envelope	This document, RL-13

#	Target	Change	Origin
18	LLD	Section 6.2: extend the TestHooks seam to idempotency (between the check and the insert), stock (between read and write) and payment, for the mutation variants CC-01m, CC-02m and CC-05m; add a test-only switch that makes ticket creation fail, active only in the test and loadtest profiles	This document, CC, RS-01
19	LLD	Section 3.7: inject a time meter into the bucket store; add roles-test.sql and a DatabaseCleaner that truncates as the schema owner	This document, 3.3, 3.4
20	Process	Export the API document to Markdown in docs/api.md on each release, because AR-07 reads its case tables	This document, AR-07

Change 16 has already been applied to LLD section 4.6.

18. Where the document set stands

Document	ID	Status
1 Product Development Document	KF-PDD-001	v0.1
2 Software Requirements Specification	KF-SRS-001	v0.1, changes 1 to 6 pending
3 System Architecture	KF-SAD-001	v0.1, changes 7 to 9 pending
4 Database Design	KF-DBD-001	v0.1, changes 10 and 11 pending
5 API Specification	KF-API-001	v0.1, changes 12 and 13 pending
6 Low-Level Design	KF-LLD-001	v0.1, changes 14 to 19 pending (16 applied)
7 Test Plan and Test Cases	KF-TPL-001	v0.1

The design phase is complete. The next work is the build: apply the change register, then follow the 19 steps of LLD section 7, writing the tests of section 16.1 first at each step.

Version	Date	Author	Change
0.1	30 October 2026	Rathi	First draft